

I. DevOps Overview**1. Traditional Software Development Models**

+ **Waterfall**: Linear, rigid, no return to previous stages. **V-Model**: Waterfall + testing mapped for each dev phase.

+ **Issues**: Slow feedback, Inflexibility to change, Late testing, High deployment risk, Wasted resources

2. What is DevOps?

Definition: DevOps = Development + Operations.

Goal: Deliver software faster, reliably, collaboratively.

Cycle Stages: Plan → Code → Build → Test → Release → Deploy → Operate → Monitor

3. DevOps Core Practices (match with challenges)

Slow delivery → **CI/CD, Automated Testing**, Poor quality → **Test Automation, Consistent Envs**, Risky deployments → **Config Management, Feature Toggles**, Dev-Ops silos → **Cross-team collaboration tools**

4. Agile Methodology

Agile = Iterative, flexible, customer-focused. **4 Core**

Values: **Individuals** over Processes, **Working**

Software over Docs, **Customer Collaboration** over

Contract, **Responding to Change** over Plans **Scrum**:

Sprint-based, roles/events/artifacts. **Kanban**: Visual workflow, WIP limits

5. (Phase/Agile/DevOps)

Planning/Sprint planning, backlog/Environment setup for deploy, **Development**/Small increments,

collab/CI, automation, **Testing**/QA + customer

feedback/Automated tests, CI pipeline,

Deployment./Continuous deployment (CD),

Maintenance/Feedback-driven

improvement/Monitoring, auto-fixes, performance logs

6. DevOps Toolchain (categorized)

CI/CD: Jenkins, GitHub Actions, GitLab CI.

Containerization: Docker, Podman. **Orchestration**:

Kubernetes, Helm, ArgoCD. **IaC**: Terraform, Pulumi,

Config Management: Ansible, Puppet, Chef.

Monitoring: Prometheus, Grafana, ELK Stack.

Collaboration: Slack, JIRA, Confluence

7. DevOps Principles

Automation everywhere, CI/CD pipelines, Monitoring and feedback loops, Infrastructure as Code (IaC), Security and Compliance as Code, Cultural shift: Dev + Ops

8. DevOps Best Practices

Shift-left testing: early & automated tests.

Immutable infrastructure: no manual changes.

Blue/Green deployments: reduce downtime.

Canary releases: gradual rollout. **Rollback strategy**:

instant recovery plan. **Observability**: logs, metrics,

traces

9. DevSecOps & Security Principles

Shift-left Security: Perform security testing early

(SAST, DAST in CI/CD). **Secrets Management**: Never

hardcode secrets. Use K8s Secrets, Vault, or env vars.

Image Scanning: Use tools like Trivy, Clair, Gype to

detect vulnerabilities. **Policy as Code**: Enforce rules

using OPA/Gatekeeper. **Security as Code**: Treat

security configs (RBAC, TLS) like versioned code.

SBOM (Software Bill of Materials): Track

dependencies → security scan

Tools: Syft, CycloneDX, Gype

10. DevOps Culture & Soft Skills

CALMS: **Culture** (Shared responsibility, no blame) –

Automation (Across build → test → deploy → infra) –

Lean (Eliminate waste, fast feedback) –

Measurement (Track DORA metrics (lead time, MTTR...)) –

Sharing (Knowledge, postmortems, CI config, dashboards). Emphasize **blameless**

postmortems, shared accountability, feedback

loops.

DevOps Anti-patterns: “DevOps is just tools”,

Manual builds, tests, deployments. Siloed teams (Dev

≠ Ops). No rollback or monitoring plan.

II. Architecture Pattern**1. Layered Architecture**

Structure: Presentation → Business Logic → Data

Access. **Pros**: *Maintainability, modularity*, clear

separation of concerns. **Cons**: *Performance overhead*

due to inter-layer communication. **Use cases**:

Enterprise apps, **management** systems. **Examples**:

Laravel, ASP.NET MVC, Django

2. Event-driven Architecture

Structure: Components interact via events, typically

via brokers. **Pros**: High scalability, real-time

responsiveness. **Cons**: Difficult to debug, requires

observability. **Use cases**: IoT, real-time streaming,

Kafka, RabbitMQ. **Keywords**: publish-subscribe,

asynchronous, event bus

3. Microkernel Architecture

Structure: **Core system** with **plug-ins**. **Pros**:

Extensible, easy feature addition. **Cons**: Core

complexity, risk of plugin mismanagement. **Use**

cases: IDEs, OS, plugin-based apps. **Examples**:

VSCode, Eclipse, Linux Kernel

4. Microservices Architecture

Structure: **Independent services** communicate via

REST/gRPC APIs. **Pros**: Scalability Tech flexibility.

CI/CD-friendly. **Cons**: Complex orchestration,

Requires distributed tracing. **Use cases**: E-commerce,

SaaS, Cloud-native systems. **Tools**: Kubernetes,

Docker, Istio, Prometheus

5. Monolithic Architecture

Structure: All logic in **one deployable artifact**. **Pros**:

Easy dev, fast local testing, simple debugging. **Cons**:

Tightly coupled, hard to scale, challenging CI/CD. **Use**

cases: Startups, internal tools, simple apps. **Red**

Flags: One failure → crash whole app, deploy = full

rebuild.

6. Serverless Architecture

Structure: Run **functions** (FaaS) in cloud; no infra

management. **Pros**: Auto-scaling, Pay-per-use, Zero

ops. **Cons**: Vendor lock-in, Cold start latency, Less

observability. **Use cases**: API gateways, event-driven

jobs, chatbots. **Platforms**: AWS Lambda, Azure

Functions, GCP Cloud Functions

III. Git & GitHub**1. Essential Git Commands**

git log Show commit history **git diff** Compare changes

git remote add origin <url> Link to remote **git**

remote -v view remote **git fetch origin** download

changes (no merge) **git pull origin main** Fetch +

merge **git push -u origin main** push + set upstream.

2. Git Merge vs Git Rebase

History: **Preserves** full branch history - **Rewrites**

commit history. **Use Case**: Shared teams, feature

integration - Clean history before pushing.

Command: git merge – git rebase. **Conflict Risk**: Less

prone – Higher (due to re-writing)

3. GitHub & Actions

GitHub ≠ Git — Git is CLI-based VCS; GitHub is cloud

hosting.

Automate: Tests, Deployments, Builds. YAML-based

workflows. Triggered by: **push, pull_request, cron**.

DevOps Tools Relation: Lightweight CI/CD →

Alternative to Jenkins/GitLab CI.

4. Git Workflow Strategies

Git Flow: Branches → main, develop, feature/*,

release/*, hotfix/*. **Trunk-Based Dev**: Short-lived

branches, frequent merges to main. **PR Reviews**: All

changes go through Pull Request with code review, CI

checks.

5. Advanced Git Commands

git tag -a v1.0 -m "msg" – Create annotated tag. **git**

stash – Save uncommitted changes. **git cherry-pick**

<commit> – Apply commit from another branch. **git**

revert <commit> – Create a new commit that undoes

previous. **git rebase -i HEAD~n** – Interactive rebase

for clean history

IV. Jenkins**1. What is Jenkins?**

Open-source automation server for CI/CD, Developed

in 2011, written in Java, Highly extensible via plugins,

Works with: Git, Docker, Kubernetes, Selenium, JUnit..

2. CI/CD Concepts

CI (Integration): Automatically build + test after code

changes. **CD (Delivery/Deploy)**: Automate

deployment to staging or production

3. Jenkins Architecture

Jenkins Master - Orchestrates all jobs, manages

agents, stores configs. **Jenkins Agent** - Executes jobs

(can be any machine). **Executor** - Thread that runs

one job at a time (on Master/Agent). **Node** - Generic

term for Master + Agents. **Distributed architecture**

supports **parallel execution & horizontal scaling**.

4. Jenkins Job Types

Freestyle Project - GUI-based job, simple

builds/tests. **Pipeline** - Groovy/YAML scripted steps

(more control). **Multibranch Pipeline** - Auto-builds

for every branch in a repo.

5. Pipeline Types, Triggers

Declarative: Simple syntax, structured, preferred for

beginners (pipeline {}).

Scripted: Advanced logic (Groovy-based), more control (node {} block

). **Poll SCM**: Cron-style (H/5 * * * *) - Check Git

periodically. **Webhook Trigger**: GitHub → Jenkins on

new push (via webhook)

6. CI/CD Rollback & Error Handling

Job Statuses: SUCCESS, FAILURE, UNSTABLE,

ABORTED. **Rollback Types**: **Manual**: Use kubectl

rollout undo, git revert, re-deploy. **Auto**: Triggered by

health check failures or smoke tests. **Failure**

Handling Tips: Use post block in Jenkins for

cleanup/alerts. Set timeout, retry, catchError in

Jenkins pipeline

7. DORA Metrics in CI/CD

Deployment Frequency: How often code is deployed

to prod. **Lead Time for Changes**: Time from commit

→ deployment. **Change Failure Rate**: % of

deployments causing failure. **MTTR (Mean Time to**

Restore): Time to recover from incident → Key to

measuring DevOps maturity

V. Docker**1. What is Docker?**

Docker is an open-source container platform for

packaging applications into isolated environments.

Faster startup, lightweight, and more **resource-**

efficient than virtual machines.

2. Key Docker Components

Docker Engine: Core service; includes daemon, API,

and CLI. **Docker Daemon**: Runs in background,

manages containers, images, networks, and volumes.

Docker CLI: Tool to interact with Docker. **Docker**

Image: Blueprint to build a container (read-only, built

via **Dockerfile**). **Docker Container**: Running

instance of an image (isolated execution). **Docker**

Registry: Storage/repo for images (public/private:

Docker Hub, ECR, GCR)

3. Dockerfile Commands

FROM: Base image. **RUN**: Execute command during

build. **COPY/ADD**: Copy files from host to container.

WORKDIR: Set working directory.

CMD/ENTRYPOINT: Command to run when

container starts. **EXPOSE**: Document container's port.

ENV: Define environment variable. **docker build -t**

<image_name>:<tag> (.)

4. Image Management Commands

docker pull <image>: Download image. **docker**

images: List local images. **docker rmi <image>**:

Remove image. **docker history**, **docker inspect**.

5. Container Lifecycle Commands

docker run -d -p 8080:80 --name mynginx nginx:
Run in background. **docker ps:** List running. **docker ps -a:** List all. **docker stop/start <container>:** Stop/start. **docker rm <container>.** **docker logs <container>.** **docker exec -it <container> /bin/bash:** Open shell. **docker container prune:** Remove stopped. **docker inspect <container|image>.** **docker top <container>:** Xem process trong container. **docker stats:** Theo dõi realtime tài nguyên. **docker compose up <service_name>**

6. Useful docker run Flags

-d: Detached mode. **--name:** Name container. **-p:** Port mapping host:container. **-v:** Volume mount host:container. **--rm:** Auto-remove after execution.

7. Docker Volumes

Used for **persistent storage** outside the container lifecycle. **Types:** *Bind Mounts* (host paths). *Named Volumes* (-v myvolume:/app). *Anonymous Volumes.* Example: **docker run -v myvol:/data busybox**

8. Docker Networking

Bridge: Default for standalone containers (NAT-based). **Host:** Container shares host network. **Overlay:** Multi-host networking (used in Docker Swarm). **Macvlan:** Assign MAC address to container. **None:** No network (completely isolated). Containers in the same **bridge** can communicate via name.

9. Docker Registry

Public: Docker Hub (default). **Private:** Self-hosted registry. Run **private registry:** docker pull registry:2
docker run -d -p 5000:5000 --name my-registry registry:2

10. Troubleshooting Commands

docker logs <container> - View container logs.
docker inspect <container> - Detailed config info.
docker exec -it <container> /bin/bash - Open interactive shell. **docker events** - Monitor real-time Docker events

Common Errors: *CrashLoopBackOff:* App crash → check logs. *ImagePullBackOff:* Invalid image/tag or auth issue. *OOMKilled:* Memory limit exceeded. Use **docker stats** to monitor resource usage

VI. K8S

1. What is Kubernetes (K8s)?

open-source container orchestration system. Built by Google, maintained by CNCF. Automates: Deployment, Scaling, Self-healing, and Networking of containers.

2. Core Benefits

Scalability (auto + manual). **Self-healing** (reschedule failed Pods). **Load Balancing** (Services). **Storage Management** (PVC, PV, StorageClass). **Security & RBAC**

3. K8s Architecture

Control Plane: API Server, Scheduler, Controller

Manager, etcd. **Worker Node:** kubelet, kube-proxy, container runtime. **Etcd:** Stores cluster config/state (key-value store). **API Server:** Frontend for control plane (REST interface). **Scheduler:** Assigns Pods to Nodes. **Controller Manager:** watch resource states, reconcile them toward the desired state (e.g., ReplicaSet, Node controller). **kubelet:** Ensures containers in a Pod are running as expected. **kube-proxy:** Handles network routing, forwards requests to correct Pod. **container runtime:** Runs containers (e.g., containerd, CRI-O, Docker).

4. Kubernetes Objects

Pod: Basic unit – one or more containers with shared resources. **ReplicaSet:** Ensures specified # of Pods are running. **Deployment:** Declarative update for Pods and ReplicaSets (rolling updates). **Service:** Stable network endpoint to access Pods. **Namespace:** Logical isolation within cluster. **ConfigMap:** Stores non-sensitive config data (env vars, files). **Secret:** Stores sensitive info (passwords, tokens, certs). **Volume:** Persistent or ephemeral shared storage in Pods. **PVC/PV:** Persistent Volume Claim / Volume abstraction. **DaemonSet:** Runs one Pod per Node (e.g., log agents). **Ingress:** HTTP/HTTPS load balancing into cluster. **HPA:** Horizontal scaling of Pods based on metrics (CPU, memory).

5. Service Types

ClusterIP: Default; internal-only access. **NodePort:** Exposes service on static port on each node. **LoadBalancer:** External load balancer (cloud only). **ExternalName:** Maps to external DNS (via CNAME)
6. Volume Types
emptyDir, hostPath, PersistentVolume, ConfigMap, Secret. Volume data **persists for Pod lifecycle, not container**

7. RBAC (Role-Based Access Control)

Role: Permissions in 1 namespace. **ClusterRole:** Permissions cluster-wide. **RoleBinding:** Binds Role to user/group. **ClusterRoleBinding:** Binds ClusterRole across all namespaces

8. Network Policies

Controls ingress/egress traffic. Filters by: labels, namespace, ports, protocol. Requires CNI plugin like Calico, Cilium, Weave.

9. Helm (Kubernetes Package Manager)

Chart: Package with templates + values. **values.yaml:** Default input for templates. **templates/:** YAML templates. **Chart.yaml:** Metadata. **helm install my-nginx bitnami/nginx -f values.yaml --namespace dev** **helm upgrade my-nginx bitnami/nginx -f values.yaml --namespace dev** **helm uninstall my-nginx --namespace dev**

10. Key kubectl Commands

get pods: List pods. **describe pod <name>:** Pod details. **logs <pod>:** view logs. **Exec -it <pod> -- /bin/bash:** shell inside pod. **Apply/delete -f *.yaml:**

apply/delete config. **config use-context <context-name>:** Switch context. **port-forward svc/<svc> 8080:80**

11. Common Pod Failure States & Debugging

CrashLoopBackOff: App crash loop → check kubectl logs, describe pod. **ImagePullBackOff:** Image tag type or no access → check image repo/auth. **Pending:** No node resources or unschedulable → check node status. **OOMKilled:** Exceeded memory limit → tune requests/limits.

Debug Flow: *kubectl describe pod, kubectl get events, kubectl logs <pod>, kubectl exec -it <pod> -- bash*

readinessProbe: The container can handle requests before being added to the Service. **livenessProbe:** The Pod will auto-restart if the probe fails

VII. Observability

1. What is Observability?

Observability = *System characteristic* (not a tool). Enables **understanding internal system states** from external outputs. Goes beyond monitoring → it's about **"telemetry everywhere"**. **Monitoring:** *Action* we perform. **Observability:** *Capability* we build

2. The 3 Pillars of Observability

Logs: Timestamped records of discrete events. **Metrics:** Numerical values over time (CPU, mem, req/s) → Good for dashboards. **Traces:** Full request journey across distributed services (critical for latency). High **Cardinality** logs/metrics (e.g., user_id, order_id) help **debug complex systems**

3. Observability vs Monitoring

Focus/Symptoms/Root cause. **System type/**Known, static/Unknown, dynamic (microservices, cloud). **Depth/**Black-box (superficial)/ White-box (introspective). **Data/**Metrics/logs (predefined)/ Logs + metrics + traces + context. **Tool Examples/**Nagios, Zabbix/Prometheus, Grafana, OpenTelemetry.

4. Advanced Use Cases

Cloud-native: Ephemeral containers, auto-scaling → need real-time insights. **Microservices:** Distributed tracing becomes critical for debugging. **Serverless:** Observability at the function level (latency, cold starts). **Dev Personas:** Embraces ODD (Observability-Driven Development). **Value Streams:** Ties Observability to TCO, ROI, and user experience. **AIOps:** Uses AI/ML to reduce noise and automate incident response

5. Tools & Ecosystem (Modern Stack)

Metrics: Prometheus, Datadog, CloudWatch. **Logs:** ELK Stack (Elasticsearch, Logstash, Kibana), Loki. **Traces:** Jaeger, Zipkin, OpenTelemetry. **Dashboards:** Grafana, New Relic. **AIOps & Insights:** Moogsoft, Dynatrace, Splunk, Honeycomb

6. AIOps&Observability-DrivenDevelopment

AIOps: Use ML to detect anomalies, predict failures, reduce noise. **Tools:** Dynatrace, Moogsoft, Splunk,

OpenTelemetry. **ODD:** Treat observability as code → logs, metrics, traces are testable. **Value:** Improve **MTTD/MTTR**, correlate metrics to user experience. **Use Cases:** Alert suppression, automated triage, smart remediation

7. Observability Metrics

MTTD: Mean Time To Detect. **MTTR:** Mean Time To Recover. SLOs/SLAs/Error **Budgets:** Boundaries of reliability objectives. Observability helps lower **MTTD/MTTR** → faster incident resolution

VIII. Deployment Strategies & GitOps/ArgoCD

1. Core Deployment Strategies

Recreate: Stops old version, then starts new → downtime expected. **Rolling Update:** Gradually replaces old with new Pods → minimal downtime. **Blue-Green:** Runs both versions side-by-side; switch traffic when ready. **Canary:** Route a small % of traffic to new version → monitor → full rollout. **Shadow:** Sends real user traffic to new version without affecting users. **A/B Testing:** Routes based on rules (e.g., user group, geography) → controlled testing.

2. What is GitOps?

GitOps = "Infrastructure as Code + Git + CD". **Git** is source of truth. **CD tool** (e.g., ArgoCD) pulls from Git and applies changes to the cluster

3. ArgoCD: GitOps Controller for Kubernetes

Application CRD: Defines app state (source repo, target cluster, sync method). **Sync Policy:** Manual / Automatic (every commit or time window). **Self-healing:** Detects drift between Git & cluster, restores declared state. **Hooks:** Supports pre-sync/post-sync actions. **RBAC:** Role-based UI/API access. **Health Checks:** Knows how to validate app health. **ArgoCD CLI / UI:** Interactive dashboard and CLI tool (argocd app sync). **Integration:** Supports Helm, Kustomize, Plain YAML, Jsonnet

4. ArgoCD Architecture

Git Repository ---> ArgoCD Repo Server (| ArgoCD Application Controller | ArgoCD API Server + UI) ---> Kubernetes Cluster

5 GitOps vs Traditional CD

Traditional CD: Push-based – CI/CD tool (e.g., Jenkins) **pushes** changes to the cluster.

GitOps: Pull-based – Git is **source of truth**; ArgoCD **pulls** and applies state → Benefits: **Self-healing, version-controlled deployments, auditable history**